Framework for building a regression model

1) Choose a function class or Hypothesis class

Eg: 1) Linear functions

2) Kernel functions

3) Decision-tree regressors

4) Neural networks.

Assume each member function $f \in \mathcal{F}$ is <u>parametric</u>

$\underline{f(x;\theta)}$ Eg: $f(x;\theta) = w^T x + w_0$ where $\theta = (w, w_0)$

parameters

$f(x;\theta) \longleftrightarrow \hat{y} \leftarrow$ predicted label.

Chosen by domain expert based on characteristic of data

2) Define an error function.

2) Error function or Loss function to compare the true label y & the predicted value $\hat{y}$.

Examples

Square error: $(y-\hat{y})^2$

Absolute value $|y-\hat{y}|$

sensitive to outliers.

Limitation of square error: blows up for large absolute y values disproportionately

clipped absolute difference: $\min(3, |y-\hat{y}|)$

3) Define the training objective

$$L(\theta) = \sum_{i=1}^{N} l(y_i ; f(x_i, \theta))^2$$

4) $\min L(\theta)$ w.r.t θ ; solve the numerical optimization problem

$$\underset{\theta}{\text{minimize}}\ L(\theta)$$

Assume $\theta \in \mathbb{R}^p$

p = # of parameters.

parameter are unconstrained reals.

$L(\theta)$ is continuous in θ.

Invoke off-the-shelf optimizer to solve this objective.

$$\hat{\theta} \leftarrow \underset{\theta}{\text{minimize}}\ L(\theta)$$

5) Ensure that chosen f with $\hat{\theta}$ generalizes to unseen examples, possibly exploring multiple $f \in \tilde{\mathcal{F}}$.

Linear Functions as Hypothesis Class:

1) First we choose a feature map or basis transform of X

$\phi(X) \rightarrow q$- dimensional real vector. $X \equiv \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}$

Examples:

1) $\phi(X) = X$ ← Linear map

$$\phi(X) = \begin{bmatrix} x_1 \\ x_1^2 \\ x_1^3 \\ \vdots \\ x_1^p \end{bmatrix}$$

$d = 1$

other examples: Gaussian kernels.

Example of hypothesis class: polynomials of degree p where p is any value from $\{1 \ldots 100\}$

$$\mathcal{F} = \{ W^T \phi(x) + W_0 : \quad \forall\ W \in \mathbb{R}^q ; \quad w_0 \in \mathbb{R} \}$$

$$W = \begin{bmatrix} w_1 \\ \vdots \\ w_q \end{bmatrix}$$

Example $q = 2$

$$\mathcal{F} \equiv \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}^T \begin{bmatrix} \varphi_1(x) \\ \varphi_2(x) \end{bmatrix} + W_0 \equiv w_1 \varphi_1(x) + w_2 \varphi_2(x) + W_0$$

$$f(x, \theta) \equiv \sum_{j=1}^{q} w_j \varphi_j(x) + w_0 \qquad \theta \equiv \{ w_1, \dots w_q, w_0 \}$$

$$= \{ W, w_0 \}$$

2) Choose error function as square error:

$$l(\hat{y}, y) = (\hat{y} - y)^2$$

3) Training objective:

$$\underbrace{\text{minimize}}_{W,\ w_0}\ \ \sum_{i=1}^{N} \left(W^T \phi(x_i) + w_0 - y_i \right)^2$$

$$\underbrace{W,\ w_0}_{\theta}$$

4) For simplicity of notation we will assume from now on that

$\phi(x) = x$; Additionally assume that x is augmented with a 0^{th} coordinate whose value is always 1:

$$\left.\begin{aligned} x &\triangleq \phi(\tilde{x}) \\ x &\triangleq \begin{bmatrix} 1 \\ \tilde{x} \end{bmatrix} \\ W &\triangleq \begin{bmatrix} w_0 \\ \tilde{W} \end{bmatrix} \end{aligned}\right\} \Rightarrow \tilde{W}^T \phi(\tilde{x}_i) + w_0 \equiv W^T x$$

optimization ~~task~~

$$\underset{W}{\text{minimize}} \ \boxed{\sum_{i=1}^{N} (W^T x_i - y_i)^2} \leftarrow L(W)$$

$W \in \mathbb{R}^{q+1}$ $\quad x \in \mathbb{R}^{q+1}$

We will rewrite in matrix notation

Recall

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{bmatrix} \qquad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$$

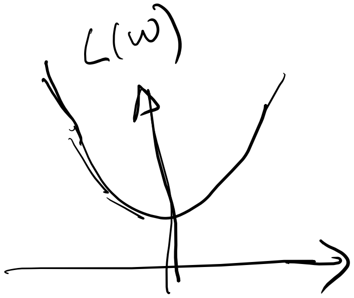
$$L(W) = \|Xw - Y\|^2$$

$$= (Xw - Y)^T (Xw - Y)$$

$$= \left((Xw)^T - Y^T\right)(Xw - Y) = \underline{w^T X^T X w - 2 w^T X^T Y + Y^T Y}$$

$$\underset{W}{\text{minimize}}\; L(W) \equiv \min_{W}\; \underline{W^T X^T X W} - 2 W^T X^T Y$$

Is $L(W)$ a convex function of W?



$q = 1$

__Homework__

Show that $L(W)$ is convex in W

Hint: Hessian of a twice differentiable function.

[Theorem 8.1.1 of PML book]

For convex functions, we can solve for the minima by equating gradient to 0.

[Section 8.1.1.1 of PML book on optimality properties]

$$\nabla_W L(W) = \nabla_W \left[\underline{W^T X^T X W} - 2 W^T X^T Y \right]$$

[Section 7.8 on matrix calculus]

$$= 2 X^T X \hat{W} - 2 X^T Y = 0$$

Let's assume that $X^T X$ is invertible – $X^T X$ is full-rank.

that is, all columns of X are linearly independent

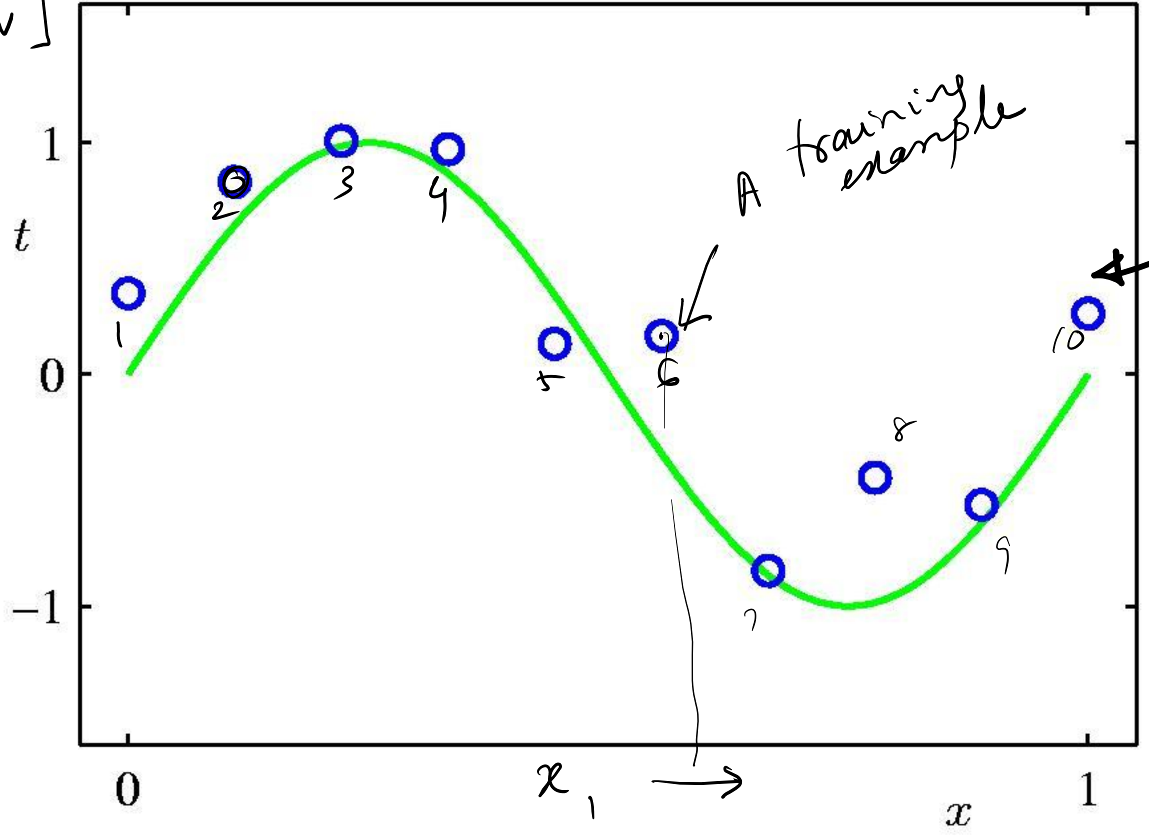
$$\hat{W} = (X^T X)^{-1} X^T Y$$

$\hat{W}$ → fitted parameter obtained by minimizing loss on the training data.

5) $\hat{W}$ may overfit the training data.
How to control overfitting & generalize to unseen example?

Regression: one-dimensional data

$\mathcal{D}$
$$\begin{bmatrix} x_i & y_i \\ \vdots & \vdots \\ x_N & y_N \end{bmatrix}$$



$\mathcal{D}$

$d = 1$

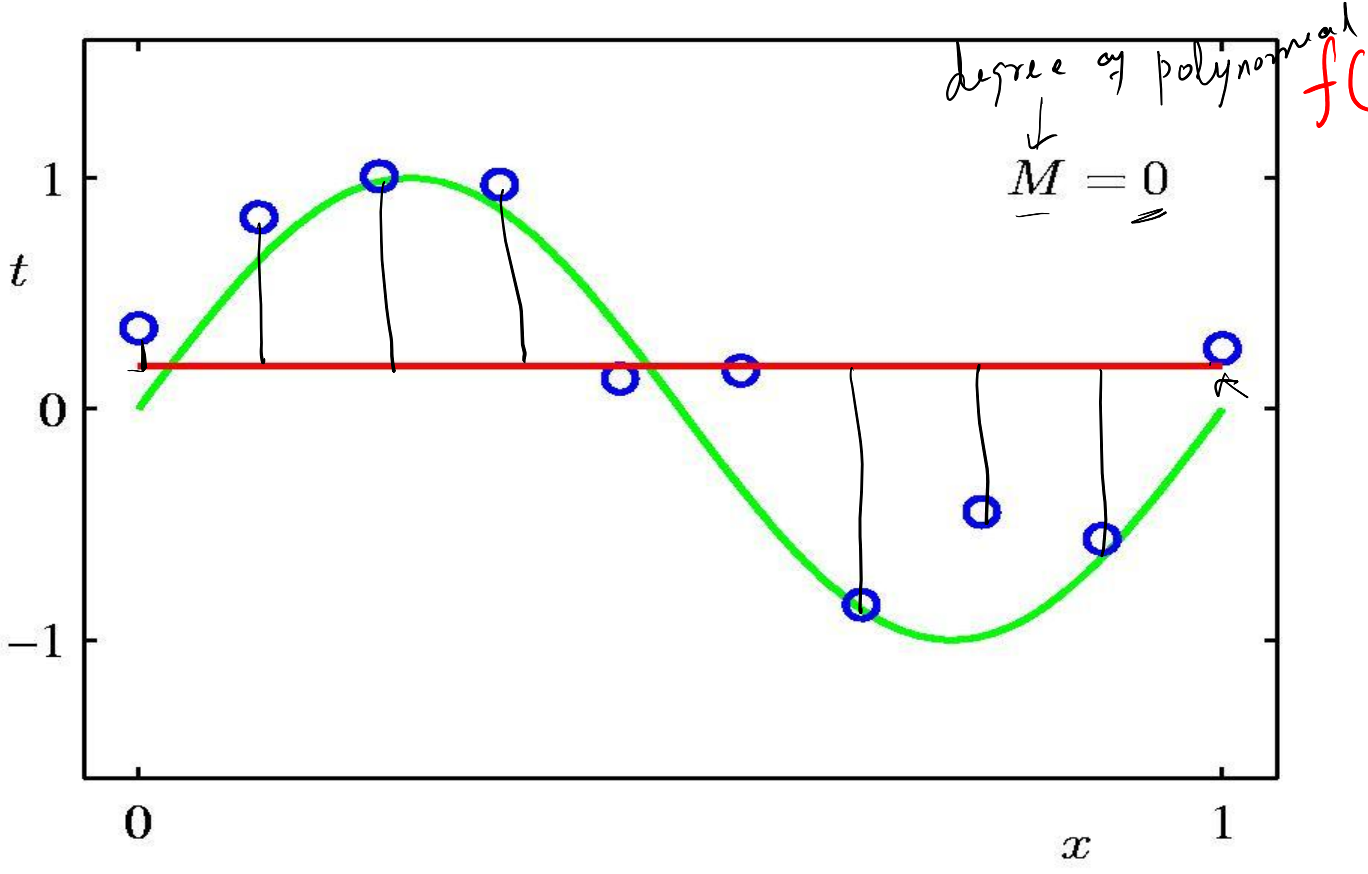
one training instance

$N = 10$

Fitting with a constant function

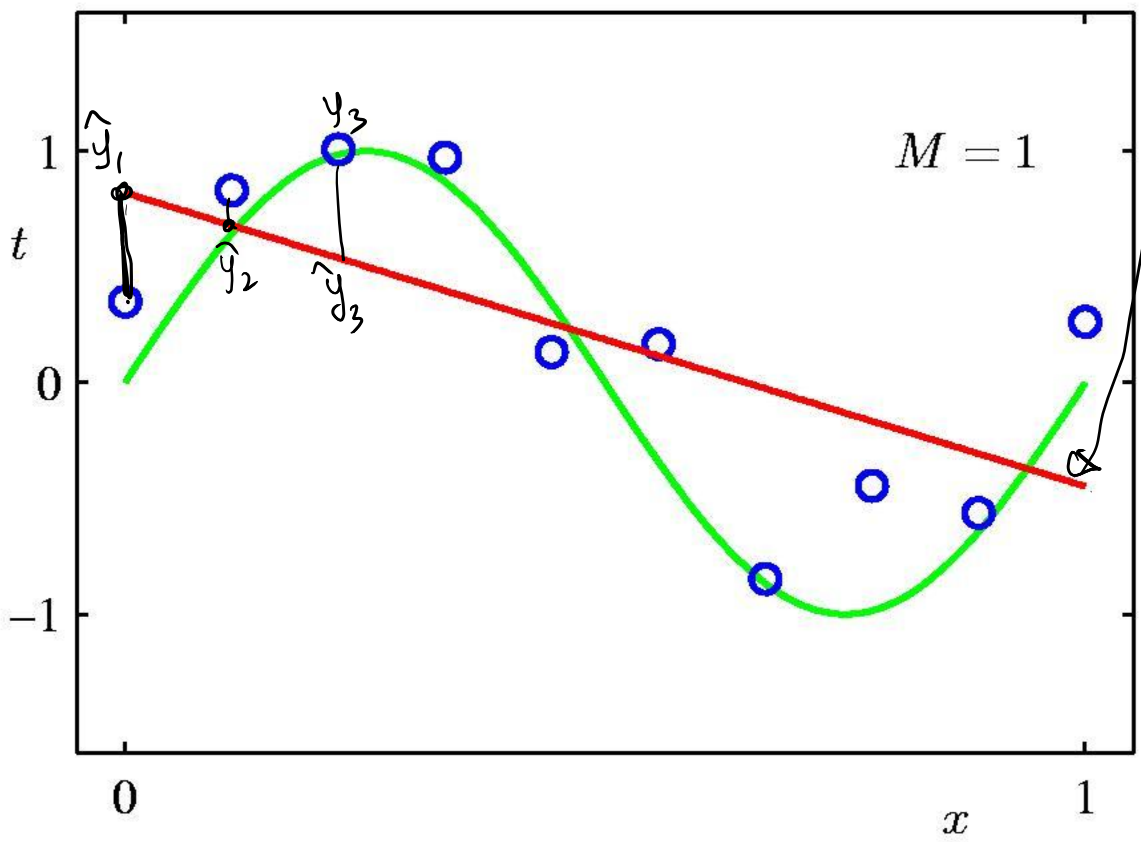
$\phi(x) = 1$

$f(\underline{x}; w) = w_0$



Fitting with a linear function $\phi(x) = x$

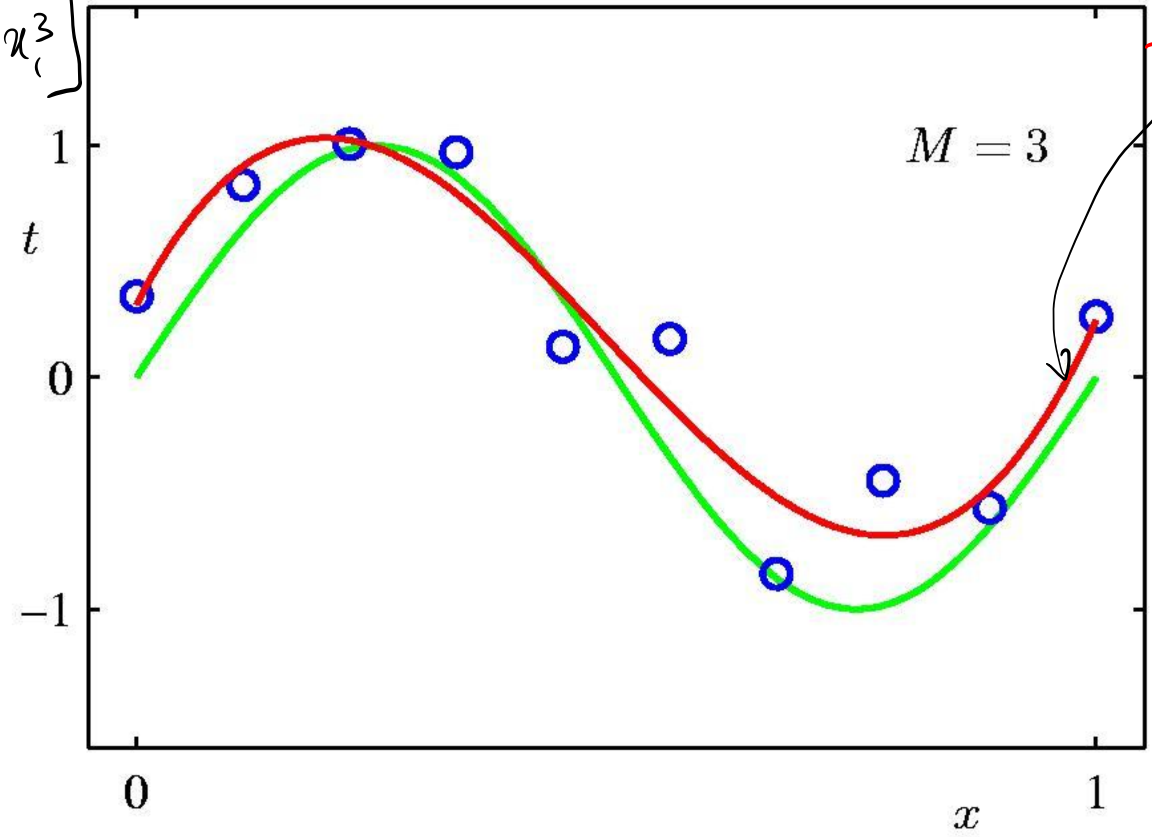
$f(x;w) = w_1 x_1 + w_0$



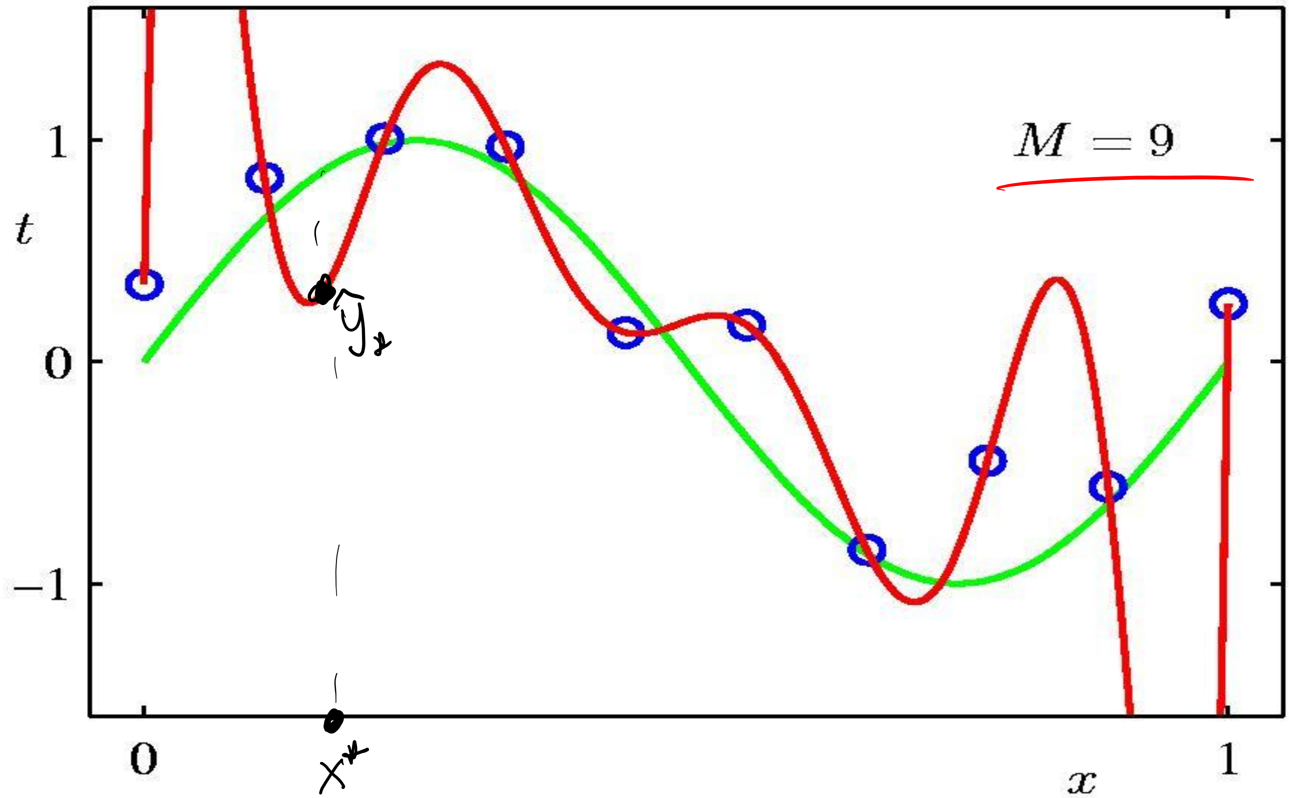
$$\varphi(x) = \begin{bmatrix} x_1 \\ x_1^2 \\ x_1^3 \end{bmatrix}$$

Fitting with a cubic function

$$f(x; w) = w_0 + w_1 x_1 + w_2 x_1^2 + w_3 x_1^3$$



Overfitting with a 9th degree polynomial



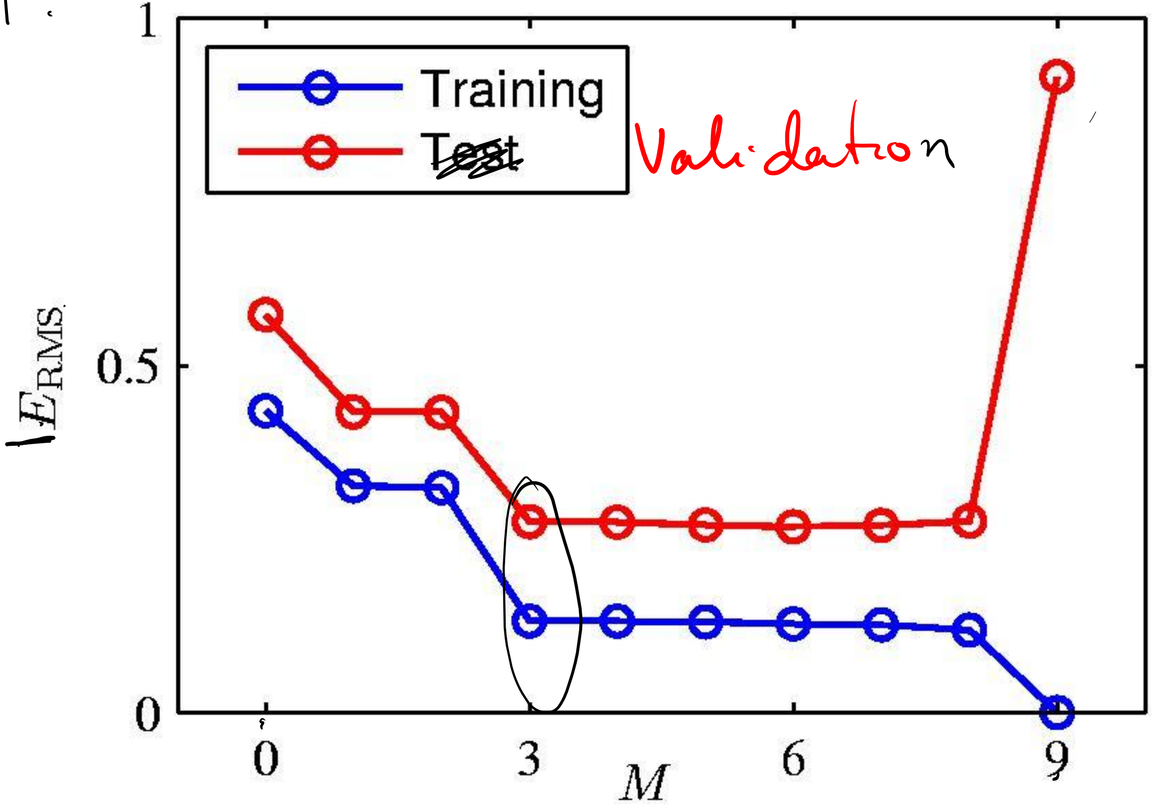
How to detect overfitting

(T) Train set

D

D−T : Validation set

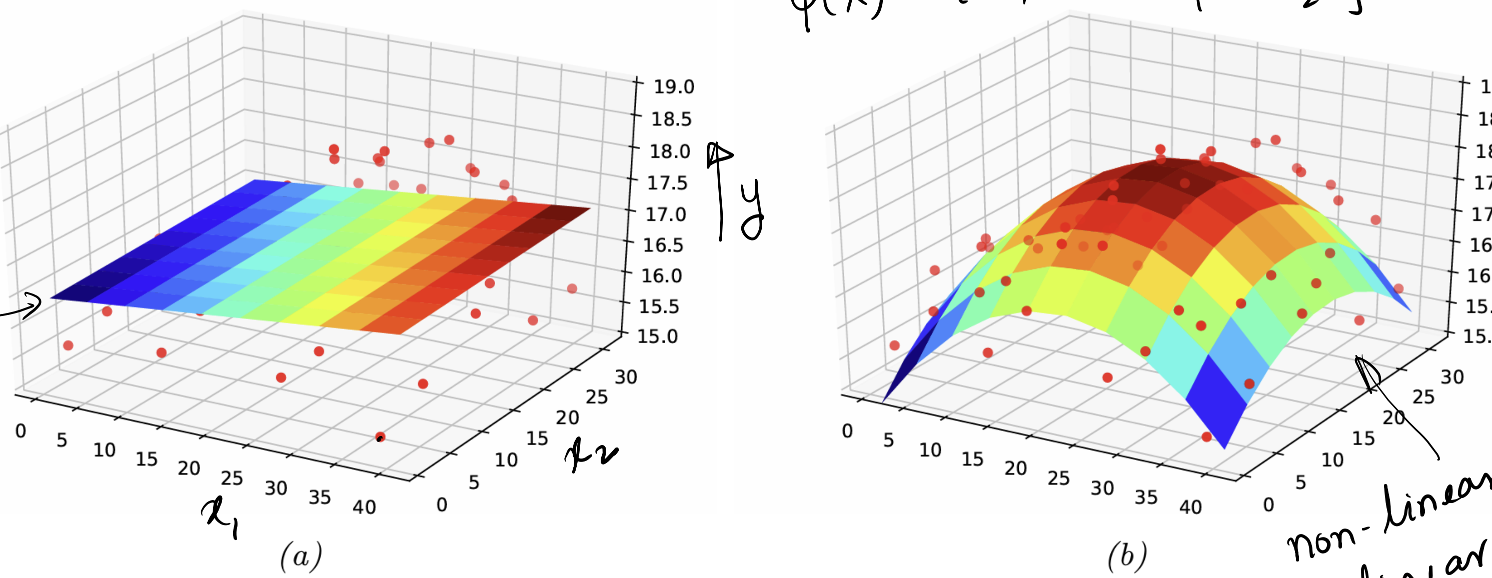
loss



Regression on 2-dimensional data

$\phi(x) = [x_1 \;\; x_2 \;\; x_1^2 \;\; x_2^2]^T$

$\phi(x) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$



y

x_1

x_2

non-linear in x

but linear in W

Figure 1.6: Linear and polynomial regression applied to 2d data. Vertical axis is temperature, horizontal axes are location within a room. Data was collected by some remote sensing motes at Intel's lab in Berkeley, CA (data courtesy of Romain Thibaux). (a) The fitted plane has the form $\hat{f}(\boldsymbol{x}) = w_0 + w_1 x_1 + w_2 x_2$. *(b) Temperature data is fitted with a quadratic of the form* $\hat{f}(\boldsymbol{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_2^2$. *Generated by linreg_2d_surface_demo.ipynb.*

How to avoid overfitting

- **Cross-validation** with unseen data

$$D \begin{cases} T & \leftarrow \text{train with } T \\ D-T & \to \text{choose hypothesis complexity at } D-T \to \text{validation set.} \end{cases}$$

If D is small, this partitioning may leave too little data for training.

Can be generalized to F-fold cross-validation.

Limitation: $\longrightarrow$ expensive, require training multiple models to select one.

Another option:

Control over-fitting during training:

when d is large, then even linear feature maps can overfit on the training data when W is unconstrained.

One popular way to control overfitting is by constraining W to take small values.

$R(W)$

1) $\|W\|_2 \leq \tau_2$ ← a constant chosen by human

   Ridge-regression

2) $\|W\|_1 \leq \tau_1$ ← L_1-norm of W should be small

   Lasso regression.

Training objective:

$$\min_{w} L(w) + \gamma R(w)$$

is equivalent to

$$\boxed{\min_{w} L(w) \quad \text{s.t.} \quad R(w) \le \tau}$$

Lagrange multiplier discussed in section 8.5 of [PML] book.

We will revisit this topic.